

CyberShip Software Suite

Architecture, Operation, and Configuration Manual

Marine Cybernetics Laboratory (MCLab)
NTNU

June 2026

Contents

1	Introduction	4
1.1	Supported Vessels	4
2	Package Overview	4
3	System Architecture	5
3.1	High-Level Architecture	5
3.2	Distributed ROS 2 Network	5
3.3	Coordinate Frames	5
3.4	Network Topology	6
4	Installation	6
5	Network and Discovery Server Configuration	7
5.1	Connecting a Workstation	7
5.2	Virtual Machine Setup	7
5.3	SSH Access	7
6	Bringing Up a Vessel	7
6.1	Automatic Startup	7
6.2	Manual Bringup	8
6.3	Bringup Include Files	8
7	Localization	8
7.1	Motion Capture Node Details	9
8	Motion Capture System	9
8.1	Startup Procedure	9
8.1.1	Quick Startup Checklist	9
8.1.2	Qualisys Track Manager Startup	10
8.1.3	Rigid Body Setup and Verification	11
8.1.4	Streaming Output Verification	12
8.2	Common Failure Modes and Symptoms	12
8.2.1	Incorrect Body Frame Origin	12

8.2.2	Low Marker Count	13
8.2.3	High Residuals	13
8.2.4	Tracking Dropouts	14
8.2.5	Camera System Mismatch	14
8.3	Screenshots and Examples	15
9	Thruster System	17
9.1	Thruster Types	17
9.2	Safety Watchdog	17
9.3	Enabling / Disabling Thrusters	18
9.4	Thruster Configuration (YAML)	18
10	Dynamic Positioning	19
10.1	Velocity Controller	19
10.2	Force Controller	19
10.3	Launch Files	19
10.4	Vessel Model Support	19
11	Simulation	19
12	Visualization	20
12.1	RViz	20
12.2	URDF / Robot State Publisher	20
12.3	Utilities Web UI	20
12.4	Force Mux GUI	20
13	Joystick / Teleoperation	20
14	ROS 2 Interfaces	21
15	External Dependencies	21
16	Configuration Reference	22
16.1	Centralized Configuration	22
16.2	Common Launch Arguments	22
16.3	ROS Environment	22
17	Systemd Service Management	22
18	Operational Procedure	23
18.1	Standard Startup Sequence	23
18.2	Troubleshooting	23
19	Developer Notes	23
19.1	Adding a New Vessel Model	23
19.2	Node Naming Convention	24
19.3	Extending Interfaces	24
20	Quick Operation Manual	24
20.1	First-Time Workstation Setup	24

20.2 Starting an Experiment	24
20.3 Running a Student Controller	25
20.4 Joystick Teleoperation	25
20.5 Shutting Down	26
20.6 Restarting the Bringup on a Vessel	26
A Quick-Reference Command Cheatsheet	26

1 Introduction

The **CyberShip Software Suite** is a collection of ROS 2 packages developed at the Marine Cybernetics Laboratory (MCLab) at NTNU. Together they provide a complete software stack for simulating, visualizing, and controlling autonomous maritime vessels. The suite covers everything from digital-twin URDF models and real-time sensor integration (IMU, motion capture) to advanced control algorithms (dynamic positioning, thrust allocation).

The suite is published on GitHub at https://github.com/NTNU-MCS/cybership_software_suite and archived on Zenodo (DOI: 10.5281/zenodo.17233653).

1.1 Supported Vessels

Three vessel models are currently supported:

Voyager	Compact vessel used for general control experiments.
Enterprise	Medium-size vessel; simulation supported.
Drillship	Large vessel with dedicated launch configuration.

2 Package Overview

The repository is organized as a collection of focused ROS 2 packages. Table 1 summarizes each package and its primary responsibility.

Package	Description
cybership_bringup	Launch files that orchestrate the complete startup of a vessel (hardware drivers, localization, sensor nodes).
cybership_simulator	Lightweight Python simulation scripts based on simplified vessel physics.
cybership_viz	RViz configurations and launch files for real-time vessel visualization.
cybership_description	URDF models, meshes, and coordinate-frame definitions for each vessel.
cybership_config	Centralized YAML configuration files shared across the suite.
cybership_interfaces	Custom ROS 2 message and service definitions for maritime operations.
cybership_controller	Base package for vessel controllers.
cybership_dp	Dynamic positioning controllers (force and velocity).
cybership_thrusters	Low-level thruster drivers supporting VSP, fixed, and azimuth types.

Package	Description
<code>cybership_mocap</code>	ROS 2 bridge to the Qualisys motion capture system.
<code>cybership_utilities</code>	Shared helper functions, launch argument definitions, and GUI tools.
<code>cybership_external</code>	Third-party submodule dependencies (IMU driver, Dynamixel, PCA9685, etc.).
<code>cybership_tests</code>	Integration and unit tests for the suite.

Table 1: CyberShip Software Suite packages.

3 System Architecture

3.1 High-Level Architecture

The system follows a layered ROS 2 architecture. Data flows from physical sensors upward through localization and state estimation to high-level control, then back down through thrust allocation to the actuators.

1. **Sensor layer** – IMU (BNO055), motion capture (Qualisys).
2. **Localization layer** – `robot_localization` EKF fuses sensor data into a consistent odometry estimate.
3. **Control layer** – Dynamic positioning controllers compute force and torque demands.
4. **Actuation layer** – The thruster package converts force demands to actuator signals (PWM, RPM, servo angle).
5. **Hardware layer** – PCA9685 PWM board, Dynamixel azimuth servos.

3.2 Distributed ROS 2 Network

The system runs as a distributed ROS 2 network across multiple machines. The **cybership-master** Raspberry Pi (192.168.1.102) acts as the **Fast DDS discovery server** for all participants. Vessels run their own bringup as systemd user services and connect back to the discovery server automatically.

Host	Role / Address
<code>cybership-master.local</code>	Discovery server, UI host – 192.168.1.102
<code>cybership-voyager.local</code>	Vessel node
<code>cybership-drillship.local</code>	Vessel node
192.168.1.10	Qualisys motion capture system (read-only)

3.3 Coordinate Frames

The vessels use two main coordinate frames defined in `cybership_description`:

base_link East-North-Up (ENU) frame. All sensors are defined here.

base_link_ned North-East-Down (NED) frame. Used for navigation and control.

The NED frame is obtained by a static transform applied to the ENU frame.

3.4 Network Topology

Figure 1 shows the physical network layout of the CyberShip lab. All machines communicate over a shared 192.168.1.1/24 Wi-Fi LAN. The **cybership-master** Raspberry Pi acts as the central hub: it runs the Fast DDS discovery server (port 11811), serves the web UI (port 8001), and hosts any lab-side ROS 2 nodes. Vessel Raspberry Pis register with the discovery server automatically at boot. Student workstations join by exporting `ROS_DISCOVERY_SERVER` in their shell environment.

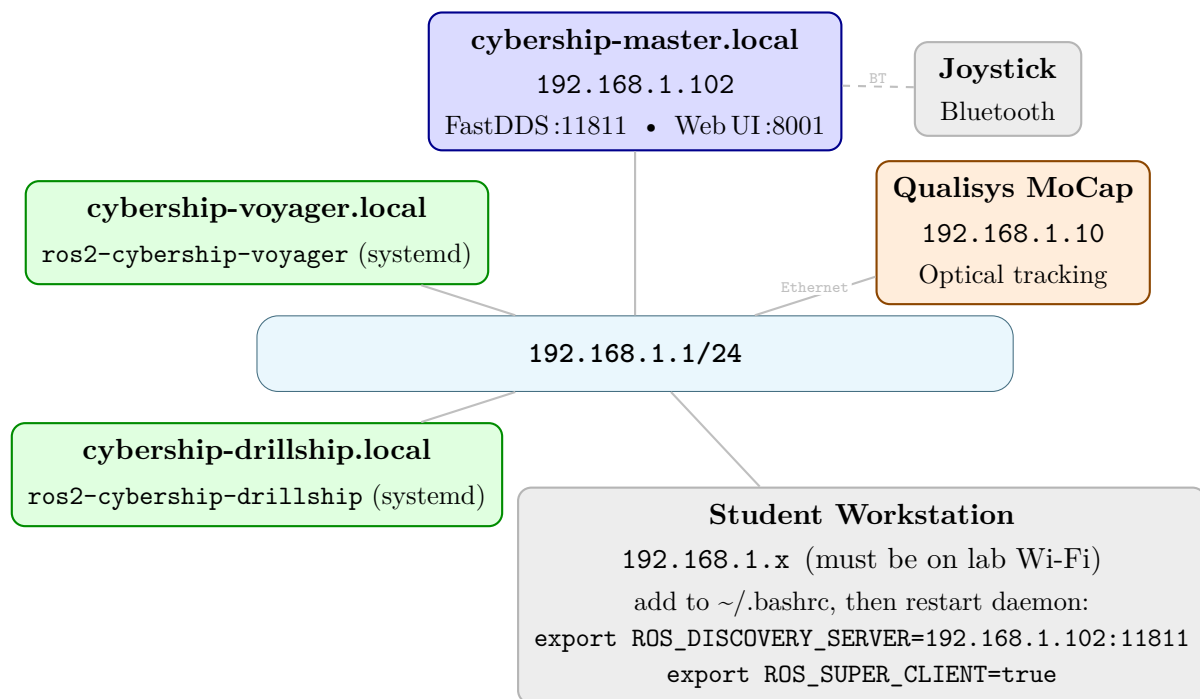


Figure 1: Physical network topology of the CyberShip lab. Vessels and workstations communicate over the 192.168.1.1/24 Wi-Fi network. **cybership-master** is connected to the Qualisys system via a dedicated Ethernet cable and acts as the Fast DDS discovery server for all ROS 2 participants. The Bluetooth joystick pairs with the master and its events are forwarded as ROS 2 topics across the network.

4 Installation

Installation instructions are maintained in the repository `README.md` and kept up to date with each release. Refer to:

https://github.com/NTNU-MCS/cybership_software_suite#installation

5 Network and Discovery Server Configuration

Note: The IP addresses listed throughout this document reflect the current lab configuration and are subject to change. If an address does not respond, verify the current assignment with the lab supervisor before attempting further troubleshooting.

The CyberShip lab uses a dedicated 192.168.1.1/24 subnet. All machines (vessels, lab computers, and the master node) must be on this subnet.

5.1 Connecting a Workstation

Add the following lines to `~/.bashrc` on any machine that needs to join the ROS 2 network:

```
export ROS_DISCOVERY_SERVER="192.168.1.102:11811"
export ROS_SUPER_CLIENT=true
```

After editing, restart the ROS 2 daemon:

```
ros2 daemon stop ; ros2 daemon start
```

5.2 Virtual Machine Setup

Students using a virtual machine must set their VM network adapter to **Bridged** mode and select the Wi-Fi adapter as the bridged interface. Verify the assigned IP is in the 192.168.1.1/24 range with `ip addr`.

5.3 SSH Access

```
ssh mclab@cybership-master.local
ssh mclab@cybership-voyager.local
ssh mclab@cybership-drillship.local
```

Tip: Always use `tmux` when working on `cybership-master` so sessions persist across disconnections.

6 Bringing Up a Vessel

6.1 Automatic Startup

Each vessel automatically starts its bringup at boot via a **systemd user service** named `ros2-cybership-<vessel_model>`. The service runs:

```
ros2 launch cybership_bringup all.launch.xml
```

You do not normally need to SSH into the vessel; simply powering it on (connecting the battery) is sufficient.

6.2 Manual Bringup

To manually launch bringup for a vessel, use the corresponding launch file from `cybership_bringup`:

```
# Voyager
ros2 launch cybership_bringup voyager.launch.py \
vessel_name:=voyager vessel_model:=voyager

# Enterprise
ros2 launch cybership_bringup enterprise.launch.py

# Drillship
ros2 launch cybership_bringup drillship.launch.py
```

6.3 Bringup Include Files

The bringup package composes several include launch files, each starting a specific subsystem:

`azimuth_controller.launch.py`

Starts the Dynamixel azimuth servo controller node.

`imu_bno055.launch.py` Starts the BNO055 IMU driver node.

`motion_capture_system_connector.launch.py`

Connects to the Qualisys motion capture server and retrieves rigid-body poses.

`motion_capture_system_transformer.launch.py`

Transforms raw mocap data into the vessel's coordinate frame.

`robot_localization.launch.py`

Runs the `robot_localization` EKF node to fuse IMU and mocap data.

`servo_driver.launch.py`

Starts the servo driver node (PCA9685-based).

`thruster_control.launch.py`

Starts the thruster controller nodes.

`urdf_description.launch.py`

Publishes the URDF model via `robot_state_publisher`.

`localization.launch.py`

Top-level localization launch combining the above components.

7 Localization

Vessel localization fuses two complementary sensor streams:

1. **BNO055 IMU** – Provides 9-DOF inertial measurements (accelerometer, gyroscope, magnetometer). Integrated via the `bno055` ROS 2 driver in `cybership_external`.

2. **Qualisys motion capture** – Provides absolute 6-DOF pose measurements at 192.168.1.10 using the `mocap4ros2_qualisys` driver and `cybership_mocap` bridge.

The fused estimate is produced by the `robot_localization` Extended Kalman Filter (EKF) node. Convergence of the EKF can be monitored by inspecting the covariance matrix on the `/<vessel_name>/measurement/odom` topic or by watching the vessel in RViz.

7.1 Motion Capture Node Details

The `cybership_mocap` package subscribes to the `rigid_bodies` topic (`mocap4r2_msgs/msg/RigidBody`) and republishes pose data as `geometry_msgs/msg/PoseWithCovarianceStamped`. Key configuration parameters are shown in Table 2.

Parameter	Type	Description	Default
<code>world_frame</code>	string	Global reference frame name	<code>mocap</code>
<code>base_frame</code>	string	Robot base frame	<code>cybership/base_link</code>
<code>rigid_body_name</code>	string	Name in mocap system	<code>cybership</code>
<code>topic_name</code>	string	Output pose topic	<code>cybership/pose</code>
<code>publish_tf</code>	bool	Broadcast TF transform	<code>true</code>

Table 2: Motion capture node configuration parameters.

Important: The order in which rigid bodies are registered in the Qualisys software determines their index. Ensure the order matches the vessel being launched.

8 Motion Capture System

8.1 Startup Procedure

This section describes the standard startup procedure for most users of the Cybership vessels in the MC Lab. If your project requires a different setup or configuration, contact the laboratory engineers before proceeding.

8.1.1 Quick Startup Checklist

Begin with this short checklist if you already know the QTM workflow:

- ☐ Open the `Qualisys_Default_MCLab` project in QTM.
- ☐ Open a new measurement window.
- ☐ Verify that all cameras are connected and that the expected number of cameras is shown.
- ☐ Check that the vessel appears in 6DOF view with the correct rigid body definition.
- ☐ Confirm that the residual is sufficiently low before starting an experiment.

8.1.2 Qualisys Track Manager Startup

Use the computer labeled *QTM Surface* (IP address: 192.168.1.10).

Open QTM (Qualisys Track Manager). When the *Manage Projects* window appears, select *Qualisys_Default_MCLab*. Press **Ctrl + N** to open a new measurement window.

Verify that the number of connected cameras displayed in the status bar matches the number of cameras currently in use (the default configuration uses seven cameras).

In the toolbar on the left, select *3D View*. This displays the laboratory coordinate system, which is defined as follows:

- The x -axis points toward the wavemaker.
- The z -axis points downward.
- The y -axis completes the right-handed coordinate system and points from the gangway toward the opposite side of the basin.
- The origin is located as close to the center of the basin as possible. At the time of writing, its exact position may vary between calibrations.
- The coordinate system orientation and origin location are established during calibration, which is normally performed by laboratory personnel. If your project requires the exact origin position or a different coordinate system orientation, contact the laboratory engineers before starting.

Press **Ctrl + D** to open the *Data Info* window. By default, this window displays marker data and coordinates.

Right-click anywhere in the window and select *Display 6DOF Data*. The window will then display the position and orientation of all defined rigid bodies:

x , y , z , roll, pitch, yaw

The displayed data also includes the *residual*¹.

Note that QTM displays position coordinates (x , y , and z) in millimeters and orientation angles (roll, pitch, and yaw) in degrees.

You may now place the vessel in the basin. If the vessel is tracked correctly in QTM and the residual is sufficiently low, the system is ready for use. Residuals may increase in regions with limited camera coverage or areas that were not fully included during calibration. The calibrated volume can be visualized by pressing **Alt + 7**.

If the vessel is not detected correctly, or the residual remains high (> 1 mm), continue to Section 8.1.3.

¹The rigid body residual is the average error, measured in millimeters, between the observed marker positions and the corresponding points in the rigid body definition. https://docs.qualisys.com/qtm/content/qtm_user_interface/qtm_windows/6dof_data_information.htm?Highlight=residual

8.1.3 Rigid Body Setup and Verification

If the vessel has not already been defined, or if you are using a different vessel, the corresponding rigid body definitions must be loaded or created.

Open the *Project Options* window by clicking the gear icon in the upper-left toolbar. Navigate to *Processing* → *6DOF Tracking*.

For the default MC Lab vessels, select *Load Bodies* in the lower-right corner and load the file

`MCLab_default.xml`

which contains the predefined rigid body definitions.

If the vessel has not previously been defined, or if you need to create a new rigid body definition, follow the procedure described below.

Place the vessel in the basin. Only one vessel should be present during rigid body acquisition. Remove other vessels from the basin or cover their markers to prevent false detections. Keep the vessel in view of as many cameras as possible; clicking *2D* on the left returns you to *camera view*, which is useful here. The initial position (x, y, z) does not matter, but make sure that the vessel heading is aligned with the desired heading corresponding to 0° yaw. If your experiment requires it, also make sure that the vessel is balanced, because QTM sets $(roll, pitch, yaw) = (0, 0, 0)$ when you acquire a rigid body.

Go to the *6DOF Settings* window in *Project Options* and press *Acquire Body*. Verify that the body has been detected, then click *Apply* and *OK*. Close the measurement window by pressing the lower of the two crosses in the top right corner. Open a new measurement window and repeat the steps until you see the rigid body coordinates in 6DOF.

For first-time users it may seem that everything is in order. The rigid body defined by the markers and its coordinates in 6DOF should both be visible, but QTM uses the centroid of the marker constellation as the body frame origin, which does not correspond to the vessel geometric center. To correct this, all Cybership vessels should have a reference marker marked by a black line on the marker stem, with its position relative to the vessel geometric center and waterline written near it. The current values at the time of writing are listed below for Voyager (Table 3) and Drillship (Table 4).

Table 3: Voyager Reference Marker Coordinates

Coordinate	Value [mm]
x	29
y	125
z	-185

Table 4: Drillship Reference Marker Coordinates

Coordinate	Value [mm]
x	158
y	187.5
z	-225

Find the marker in QTM corresponding to the reference marker on the vessel. Select the marker and note its label. The label will typically have the format **Body1-x**, although it may instead appear as **Voyager-y** or a similar name if the rigid body has previously been renamed. Return to *6DOF Tracking* settings, select the rigid body, and click *Translate*. Select the option:

So that point $\langle LABEL \rangle$ in the body has local coordinates (in mm):

Enter the values for the vessel in question and press *OK*, then press *Apply* and verify that the body origin has moved to the correct location.

8.1.4 Streaming Output Verification

After QTM is configured, confirm that the 6DOF data stream is visible to downstream ROS 2 nodes. Verify that pose updates are being published and that the vessel appears at the expected position in RViz or any other consumer of the mocap topic. If no data appears, re-check the rigid body definition, the QTM project, and the network connection to the mocap computer.

8.2 Common Failure Modes and Symptoms

8.2.1 Incorrect Body Frame Origin

Symptoms

- The vessel appears offset from its expected position in the global reference frame.
- Reported position does not correspond to the physical location of the vessel.
- Navigation, guidance, or control software behaves unexpectedly despite correct tracking quality.
- Position appears shifted relative to other tracked objects.

Possible Causes

- The rigid body origin in QTM was defined at the centroid of the marker constellation instead of being translated to the desired vessel reference point.
- The reference marker coordinates were entered incorrectly.
- The rigid body definition was recreated or modified without applying the required translation.

Recommended Actions

- Verify the location of the rigid body origin in QTM.
- Use the vessel reference marker whose position relative to the vessel's geometric center and waterline is known.
- In the 6DOF tracking settings, use the *Translate* function and assign the reference marker its known coordinates (x, y, z) in the vessel reference frame.

8.2.2 Low Marker Count

Symptoms

- Fewer markers than expected are visible in QTM.
- 6DOF tracking is unstable or unavailable.

Possible Causes

- Cameras are blocked by personnel or equipment.
- Markers are covered by water spray or reflections.
- One or more markers have detached from the vessel.
- Poor camera exposure or marker threshold settings.

Recommended Actions

- Verify that all markers are attached and clearly visible.
- Remove any obstructions from the camera field of view.
- Check the camera images for poor contrast and marker threshold settings (This option is for advanced users) or excessive reflections.

8.2.3 High Residuals

Symptoms

- Residual values exceed the normal operating range (≈ 1 mm).
- Position and orientation estimates appear noisy.
- Sudden jumps occur in the measured pose.

Possible Causes

- Markers have shifted due to a collision or other physical impact.
- Incorrect rigid body definition.
- Marker misidentification.
- Inadequate camera coverage.
- Calibration inaccuracies.

Recommended Actions

- Verify that the correct rigid body definition is loaded.
- Check that all markers are correctly identified.
- Move the vessel to an area with better camera coverage.
- Remove the rigid body from 6DOF tracking and reacquire it.

8.2.4 Tracking Dropouts

Symptoms

- The vessel disappears from the 6DOF data stream.
- Position data contains gaps.
- The body repeatedly loses and regains tracking.

Possible Causes

- Too few markers are visible to the camera system.
- Temporary obstruction by personnel or equipment.
- The vessel have strayed to the boundary of the covered volume.

Recommended Actions

- Check marker visibility in the 3D view.
- Ensure that at least three markers remain visible to the system.
- Remove obstacles from the capture volume.

8.2.5 Camera System Mismatch

Symptoms

- The number of connected cameras differs from the expected configuration(7 cameras).
- One or more camera views are missing.
- QTM displays camera connection warnings.

Possible Causes

- A camera is powered off.
- Network or synchronization cables are disconnected.
- An incorrect QTM project was loaded.
- A camera may be experiencing network or synchronization issues. In this case, contact laboratory personnel.

Recommended Actions

- Confirm that the correct project is loaded.
- Verify that all cameras are powered and connected.
- Contact laboratory personnel if a camera remains unavailable.

8.3 Screenshots and Examples

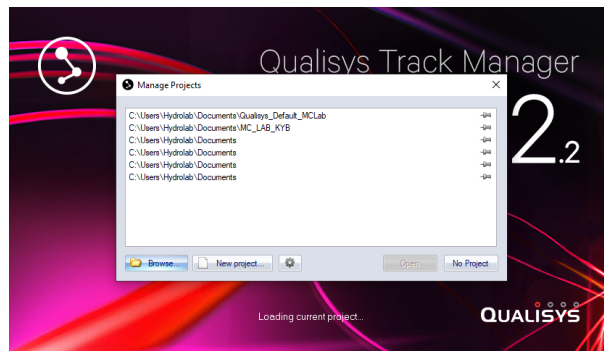


Figure 2: The first window that appears when starting QTM. Here you will select your project or create a new one



Figure 3: Start a new measurement window by clicking the white sheet in the top left or pressing CTRL + N. You must also repeat this if the qualisys system crashes.

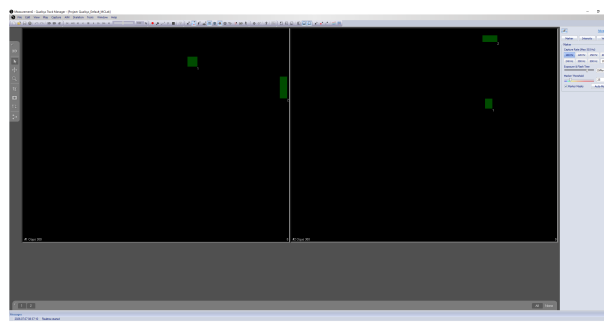


Figure 4: Camera view in *marker mode* click on the *3D* icon on the left to enter *3D mode*.

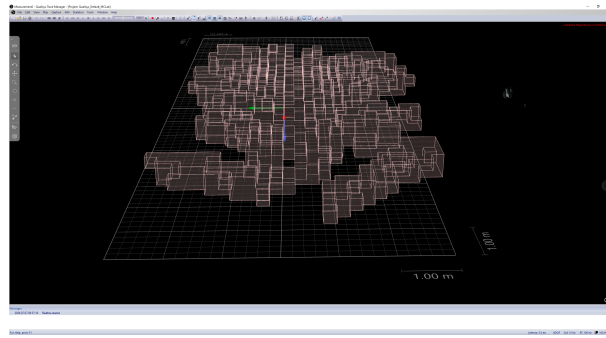


Figure 5: Lab frame of reference with the calibrated volume shown. Press CTRL + D or alternatively click *view* on the top toolbar and then select *Data Info 1* to show marker positions in $2D$.

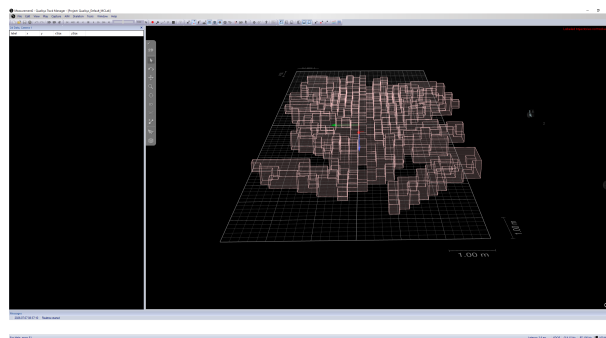


Figure 6: Lab frame of reference with the calibrated volume and 2D marker position shown. Right click and select *6DOF* to show rigid body coordinates.

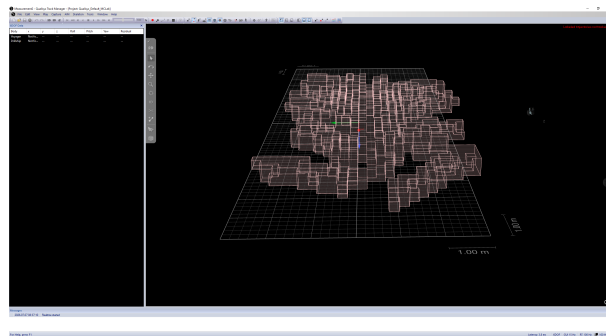


Figure 7: Lab frame of reference with the calibrated volume and rigid body coordinates.

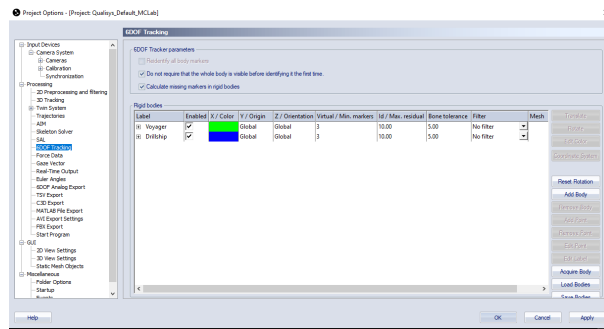


Figure 8: *6DOF Settings* window where you can *Add*, *Acquire*, *Load*, and *Save* rigid bodies.

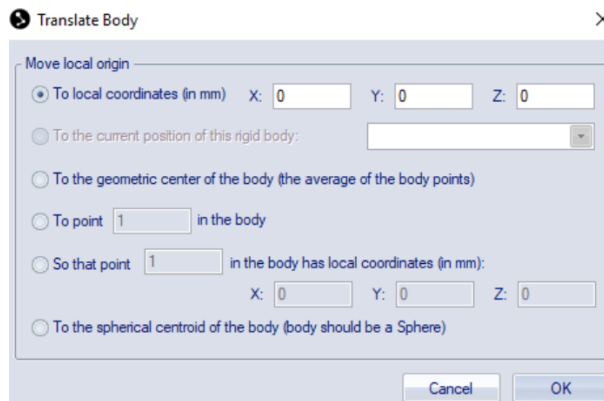


Figure 9: *Translate Body* Settings where we can move the local origin.

9 Thruster System

The `cybership_thrusters` package implements low-level thruster drivers in C++. It supports three actuator types.

9.1 Thruster Types

VSP *Voith Schneider Propeller* – Outputs differential arm positions (`arm_x`, `arm_y`) and a fixed RPM command. Suitable for omnidirectional propulsion.

Fixed A single-axis thruster outputting a normalized force signal. Force is linearly interpolated between `force_min` and `force_max`.

Azimuth Computes both a rotation angle (via `atan2`) and an RPM from the magnitude of the input force vector. Used for azimuth-steerable pods.

9.2 Safety Watchdog

All thruster controllers include a watchdog timer. If no force command is received on the configured `force_topic` within **2.0 seconds** (configurable per thruster), the driver automatically publishes zero on all output topics. The watchdog runs at 10 Hz.

9.3 Enabling / Disabling Thrusters

Thrusters are **disabled by default**. They must be explicitly enabled via ROS 2 service call:

```
# Enable
ros2 service call <vessel_name>/thruster/enable std_srvs/srv/Empty {}

# Disable
ros2 service call <vessel_name>/thruster/disable std_srvs/srv/Empty {}
```

Operational note: Before handing control to students, always navigate to the thrust allocation web UI at <http://192.168.1.102:8001> and **deactivate the thrust allocation** until the student controller is ready.

9.4 Thruster Configuration (YAML)

Thrusters are configured under the `thrusters` namespace in a YAML parameter file. Each thruster has a unique name and a type-specific set of parameters:

```
thrusters:
  bow_thruster:
    type: "fixed"
    force_topic: "/voyager/bow_thruster/force"
    signal_topic: "/voyager/bow_thruster/signal"
    force_max: 5.0
    force_min: -5.0
    signal_inverted: false
    safety_timeout: 2.0

  port_azimuth:
    type: "azimuth"
    force_topic: "/voyager/port_azimuth/force"
    angle_topic: "/voyager/port_azimuth/angle"
    rpm_topic: "/voyager/port_azimuth/rpm"
    force_max: 20.0
    force_min: -20.0
    angle_inverted: false
    rpm_inverted: false

  vsp_main:
    type: "vsp"
    force_topic: "/voyager/vsp/force"
    arm_x_topic: "/voyager/vsp/arm_x"
    arm_y_topic: "/voyager/vsp/arm_y"
    rpm_topic: "/voyager/vsp/rpm"
    force_max: 10.0
    force_min: -10.0
    rpm_cmd: 1000.0
    arm_x_inverted: false
    arm_y_inverted: false
```

10 Dynamic Positioning

The `cybership_dp` package implements two layered controllers for dynamic positioning.

10.1 Velocity Controller

The velocity controller receives a desired velocity setpoint and computes force/torque demands using independent PID loops for surge, sway, and yaw. Gains are specified in the package's `config/` YAML files. Computed demands are published to the force controller.

10.2 Force Controller

The force controller uses the **Skadi** library (<https://github.com/incebellipipo/skadipy>) for optimal thrust allocation. It receives 3-DOF force/torque demands (surge, sway, yaw) and distributes them across the available thrusters.

10.3 Launch Files

```
# Full DP system (velocity + force controller)
ros2 launch cybership_dp dp.launch.py \
vessel_name:=voyager vessel_model:=voyager

# Force controller only
ros2 launch cybership_dp force_controller.launch.py \
vessel_name:=voyager vessel_model:=voyager

# Velocity controller only
ros2 launch cybership_dp velocity_controller.launch.py \
vessel_name:=voyager vessel_model:=voyager
```

10.4 Vessel Model Support

The `voyager` model is fully supported. Support for `enterprise` and `drillship` is planned but not yet complete.

11 Simulation

The `cybership_simulator` package provides lightweight Python scripts that simulate vessel dynamics using simplified physics. Each vessel has its own dedicated script, currently including:

- CyberShip Enterprise 1 (3-DOF planar model)

The simulator publishes the same topics as the physical bringup stack (odometry, transforms) so that all higher-level control packages are compatible with both modes.

Pass `use_sim_time:=true` to any launch file when running in simulation:

```
ros2 launch cybership_bringup voyager.launch.py \
vessel_name:=voyager vessel_model:=voyager \
```

```
use_sim_time:=true
```

12 Visualization

12.1 RViz

The `cybership_viz` package provides preconfigured RViz scenes showing:

- The vessel URDF model with live TF transforms.
- Sensor data overlays (IMU, pose estimates).
- Trajectory history.

12.2 URDF / Robot State Publisher

The vessel's 3D model is published by the `robot_state_publisher` node. URDF files and mesh assets live in `cybership_description/urdf/` and `cybership_description/meshes/` respectively. To launch standalone visualization:

```
ros2 launch cybership_description description.launch.py
```

12.3 Utilities Web UI

A lightweight web interface runs on the master node and is accessible at:

<http://192.168.1.102:8001>

It can be started with:

```
ros2 launch cybership_utilities ui.launch.py
```

12.4 Force Mux GUI

The `cybership_utilities` package also includes a PyQt GUI (`nodes/force_mux_gui.py`) for inspecting and switching the active force-mux input topic for a selected vessel namespace. The namespace can be supplied via the `VEHICLE_NS` environment variable or typed directly in the GUI.

13 Joystick / Teleoperation

Bluetooth joysticks are connected to the master node using `bluetoothctl`. Once connected, the teleop stack is launched with:

```
# Drillship on joystick 0
ros2 launch cybership_teleop allocation.launch.py \
vessel_name:=drillship joy_id:=0

# Voyager on joystick 1
ros2 launch cybership_teleop allocation.launch.py \
```

```
vessel_name:=voyager joy_id:=1
```

Raw joystick events are published by the standard joy node:

```
ros2 run joy joy_node
```

14 ROS 2 Interfaces

The `cybership_interfaces` package defines the custom ROS 2 interfaces used across the suite. Currently defined interfaces are listed in Table 5.

Type	Name	Description
Service	ResetSimulator	Resets the simulator to a given 2D pose. Request: <code>geometry_msgs/Pose2D</code> . Response: <code>bool success</code> .

Table 5: Custom ROS 2 interfaces.

To depend on these interfaces from another package:

```
<!-- package.xml -->
<depend>cybership_interfaces</depend>
```

```
# Python usage
from cybership_interfaces.srv import ResetSimulator
```

15 External Dependencies

Third-party packages are included as Git submodules under `cybership_external/`. Table 6 lists each dependency.

Package	Description
<code>mocap4ros2_qualisys</code>	ROS 2 driver for the Qualisys optical motion capture system.
<code>dynamixel_azimuth_driver</code>	ROS 2 driver for Dynamixel motors used as azimuth actuators.
<code>ros2_pca9685</code>	ROS 2 driver for the PCA9685 I ² C PWM controller board.
<code>mocap4r2_msgs</code>	ROS 2 message definitions for motion capture data.
<code>bno055</code>	ROS 2 driver for the Bosch BNO055 9-axis IMU.

Table 6: External submodule dependencies.

To update all submodules to their latest upstream versions:

```
git submodule update --remote
```

16 Configuration Reference

16.1 Centralized Configuration

The `cybership_config` package stores YAML configuration files that are installed and shared across the suite. Vessel-specific overrides are placed in per-vessel subdirectories within `config/`.

16.2 Common Launch Arguments

The `cybership_utilities` package defines a set of standardized launch arguments used by all launch files:

vessel_model Model identifier string (e.g., `voyager`, `enterprise`, `drillship`).

vessel_name ROS 2 namespace for the vessel (typically matches `vessel_model`).

param_file Path to an optional override parameter file.

use_sim_time Boolean flag to use simulation time clock.

16.3 ROS Environment

The vessel's ROS setup script is located at `~/.rosrc`. Source it before running ROS 2 commands on the vessel:

```
source ~/.rosrc
```

17 Systemd Service Management

Vessel bringup and the Fast DDS discovery server are managed as **systemd user services**.

ros2-cybership-<model>

Per-vessel bringup service. Starts automatically at boot on the corresponding Raspberry Pi.

FastDDS service

Runs on `cybership-master` as a user service, providing the DDS discovery server on port 11811.

Useful service management commands:

```
# Check status
systemctl --user status ros2-cybership-voyager

# Restart bringup
systemctl --user restart ros2-cybership-voyager

# View logs
journalctl --user -u ros2-cybership-voyager -f
```

18 Operational Procedure

18.1 Standard Startup Sequence

1. Power the vessel by connecting the battery. The bringup service starts automatically.
2. On the control workstation, ensure the ROS discovery server variables are set in `~/.bashrc` (Section 5) and restart the daemon.
3. Open the web UI at `http://192.168.1.102:8001` and **deactivate the thrust allocation**.
4. Wait for the EKF to converge (monitor covariance on `/<vessel_name>/measurement/odom` or watch RViz).
5. Once the student controller is ready, re-enable thrust allocation from the web UI or via the service call.
6. Keep the vessel within the motion-capture calibrated area during experiments.

18.2 Troubleshooting

Topics not visible Restart the ROS 2 daemon:

```
ros2 daemon stop ; ros2 daemon start
```

EKF diverging Check the covariance on `/<vessel_name>/measurement/odom`. Ensure the vessel is within the mocap calibrated area and that the Qualisys system has acquired all markers.

Rigid body order mismatch

Verify the rigid body order in the Qualisys software matches the expected vessel index. Order matters for index-based assignment.

Thrusters unresponsive

Confirm they are enabled:

```
ros2 service call <vessel_name>/thruster/enable
↪ std_srvs/srv/Empty {}
```

Also verify no safety watchdog timeout has silenced the commands (check that force commands arrive within 2 s).

Discovery issues Confirm the FastDDS service is running on `cybership-master` and that your workstation is on the `192.168.1.1/24` subnet.

19 Developer Notes

19.1 Adding a New Vessel Model

1. Add URDF and mesh files to `cybership_description/urdf/` and `cybership_description/meshes/`.
2. Create a vessel-specific configuration directory under `cybership_config/`.

3. Add a new `<model>.launch.py` in `cybership_bringup/launch/`.
4. Register the model string in `cybership_utilities/utilities.py`.
5. Implement force and velocity controllers in `cybership_dp/` for the new model.

19.2 Node Naming Convention

All nodes and topics are namespaced by the vessel name to allow multiple vessels on the same ROS 2 network without collisions:

```
/<vessel_name>/<subsystem>/<topic>
```

19.3 Extending Interfaces

Custom messages and services should be added to `cybership_interfaces`. After adding `.msg` or `.srv` files, update the `CMakeLists.txt` accordingly and rebuild the package.

20 Quick Operation Manual

This section is a concise step-by-step guide for the most common lab scenarios. For full explanations refer to the relevant sections of this document.

20.1 First-Time Workstation Setup

Do this **once** on any computer that will interact with the vessels.

1. Connect to the lab Wi-Fi and confirm your IP is in `192.168.1.1/24`:

```
ip addr
```

2. Add the following lines to `~/.bashrc`:

```
export ROS_DISCOVERY_SERVER="192.168.1.102:11811"
export ROS_SUPER_CLIENT=true
```

3. Apply the changes and restart the ROS 2 daemon:

```
source ~/.bashrc
ros2 daemon stop ; ros2 daemon start
```

4. Verify you can see ROS 2 topics from the network:

```
ros2 topic list
```

20.2 Starting an Experiment

1. **Power the vessel.** Connect the battery. The vessel boots and starts the ROS 2 bringup automatically via `systemd`. No SSH needed.
2. **Open the web UI** at `http://192.168.1.102:8001` and configure the vessel according to your needs.

3. **Wait for the EKF to converge.** Watch the vessel in RViz or check the covariance on the odometry topic:

```
ros2 topic echo /voyager/measurement/odom
```

The diagonal elements of the covariance matrix should drop to small, stable values.

4. **Ensure the vessel is inside the motion-capture calibrated area** before running any controller.
5. **Launch the visualization** on your workstation (optional but recommended):

```
ros2 launch cybership_utilities ui.launch.py
```

20.3 Running a Student Controller

1. Build and source the student package on the workstation:

```
cd ~/ros_ws  
colcon build --packages-select <student_package>  
source install/setup.bash
```

2. Launch the student controller:

```
ros2 launch <student_package> <launch_file>.launch.py \  
vessel_name:=voyager vessel_model:=voyager
```

3. Verify the controller is publishing on the expected force topic, then **activate** thrust allocation from the web UI at <http://192.168.1.102:8001>.
4. Monitor the vessel. If behaviour is unexpected, immediately **deactivate** thrust allocation from the web UI.

20.4 Joystick Teleoperation

1. On cybership-master, connect the joystick via Bluetooth:

```
bluetoothctl  
# Inside bluetoothctl:  
scan on  
pair <MAC>  
connect <MAC>
```

2. Launch the joy node and teleop allocator (on master or your workstation):

```
ros2 run joy joy_node &  
ros2 launch cybership_teleop allocation.launch.py \  
vessel_name:=voyager joy_id:=1
```

3. Activate thrust allocation from the web UI and operate the vessel with the joystick.

20.5 Shutting Down

1. Stop the student controller process (Ctrl+C in its terminal).
2. Disconnect the vessel battery to power it off. The systemd service will stop cleanly.
3. No further action is needed on `cybership-master`; it can be left running between sessions.

20.6 Restarting the Bringup on a Vessel

If the vessel bringup needs to be restarted (e.g. after a configuration change):

```
ssh mclab@cybership-voyager.local
systemctl --user restart ros2-cybership-voyager
# Check it came back up:
systemctl --user status ros2-cybership-voyager
```

A Quick-Reference Command Cheatsheet

```
## Environment setup (add to ~/.bashrc)
export ROS_DISCOVERY_SERVER="192.168.1.102:11811"
export ROS_SUPER_CLIENT=true

## SSH
ssh mclab@cybership-master.local
ssh mclab@cybership-voyager.local
ssh mclab@cybership-drillship.local

## Vessel bringup (manual)
ros2 launch cybership_bringup voyager.launch.py \
vessel_name:=voyager vessel_model:=voyager

## Utilities UI
ros2 launch cybership_utilities ui.launch.py
# Web UI: http://192.168.1.102:8001

## Dynamic positioning
ros2 launch cybership_dp dp.launch.py \
vessel_name:=voyager vessel_model:=voyager

## Teleop (joysticks)
ros2 run joy joy_node
ros2 launch cybership_teleop allocation.launch.py \
vessel_name:=voyager joy_id:=1

## Thrusters
ros2 service call voyager/thruster/enable std_srvs/srv/Empty {}
ros2 service call voyager/thruster/disable std_srvs/srv/Empty {}

## Restart ROS daemon
```

```
ros2 daemon stop ; ros2 daemon start

## Monitor EKF
ros2 topic echo /voyager/measurement/odom
```